



# **Votre Aventure Docker**

## **Du Conteneur Éphémère au Développement Fluide**

Un guide visuel pour maîtriser les concepts fondamentaux de Docker :  
conteneurs, persistance des données avec les volumes, et flux de travail de  
développement avec les bind mounts.

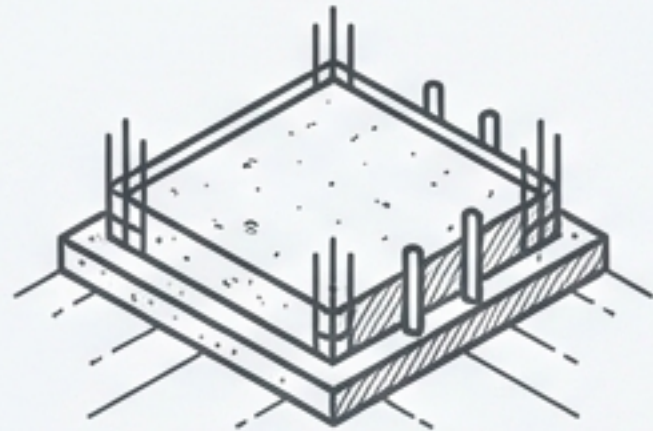


# Notre Itinéraire : Trois Étapes vers la Maîtrise



## Lancement de l'Exploration

Déployer notre premier conteneur Apache/PHP.  
Une structure simple et rapide, mais éphémère.



## Établir une Base Permanente

Assurer la survie de nos données avec les volumes Docker.  
Construire sur des fondations solides.



## Construire le Pont de Développement

Connecter notre environnement local au conteneur  
pour un développement en temps réel.



# CHAPITRE 1 : Lancement de l'Exploration

## Déployer un conteneur Apache/PHP simple

### Objectifs

- 🎯 Lancer un conteneur en tâche de fond.
- 🎯 Obtenir un terminal à l'intérieur du conteneur.
- 🎯 Créer une page PHP et y accéder depuis un navigateur.
- 🎯 Comprendre la nature éphémère d'un conteneur de base.





# Anatomie de la Commande `docker run`

**docker run:** La commande fondamentale pour créer et démarrer un nouveau conteneur.

**-p 8080:80: Publier les ports.** Mappe le port `8080` de la machine hôte au port `80` du conteneur.

```
docker run -d --name mon-php -p 8080:80 php:8.2-apache
```

**-d: Detached Mode.** Lance le conteneur en arrière-plan et affiche l'ID du conteneur.

**--name mon-php: Nommer le conteneur.** Attribue un nom lisible (`mon-php`) pour s'y référer facilement.

**php:8.2-apache: L'image Docker.** Le plan utilisé pour construire notre conteneur (PHP 8.2 avec Apache).



# Créer notre première page web dans le conteneur

## Étape 1: Ouvrir un shell interactif

Pour travailler à l'intérieur de notre conteneur qui tourne en arrière-plan.

```
docker exec -it mon-php bash
```

## Étape 2: Installer un éditeur et créer le fichier

Nous utilisons `nano` pour créer notre page `index.php` dans le répertoire web d'Apache.

```
# Dans le conteneur
apt update
apt install -y nano
nano /var/www/html/index.php
```

## Étape 3: Ajouter le contenu PHP

Collez ce code dans `nano` et sauvegardez.

```
<?php
echo '<h1>Bonjour depuis Docker 🐳 </h1>';
echo '<p>PHP fonctionne dans le conteneur !</p>';
?>
```

## Étape 4: Recharger Apache

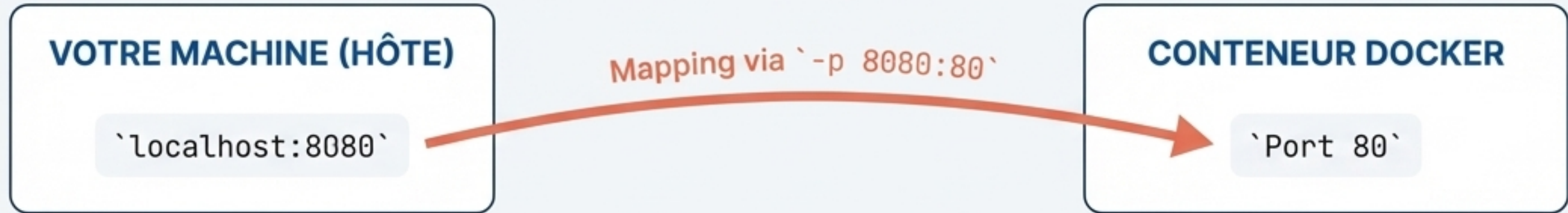
Pour que le serveur prenne en compte notre nouvelle page.

```
apache2ctl -k graceful
```



# Accéder à notre site depuis le navigateur

Le concept du Port Mapping



## Action:

Ouvrez votre navigateur et allez à l'adresse suivante :

`http://localhost:8080`

## Résultat Attendu:





# Le Caractère Éphémère d'un Conteneur

Que se passe-t-il si nous supprimons notre conteneur?

1. Quitter le shell du conteneur

```
exit
```

2. Arrêter le conteneur

```
docker stop mon-php
```

3. Supprimer définitivement le conteneur

```
docker rm mon-php
```



**Tout le travail est perdu.** Notre fichier `index.php` et les modifications faites à l'intérieur du conteneur ont disparu avec lui.

Comment pouvons-nous sauvegarder notre travail de manière permanente?

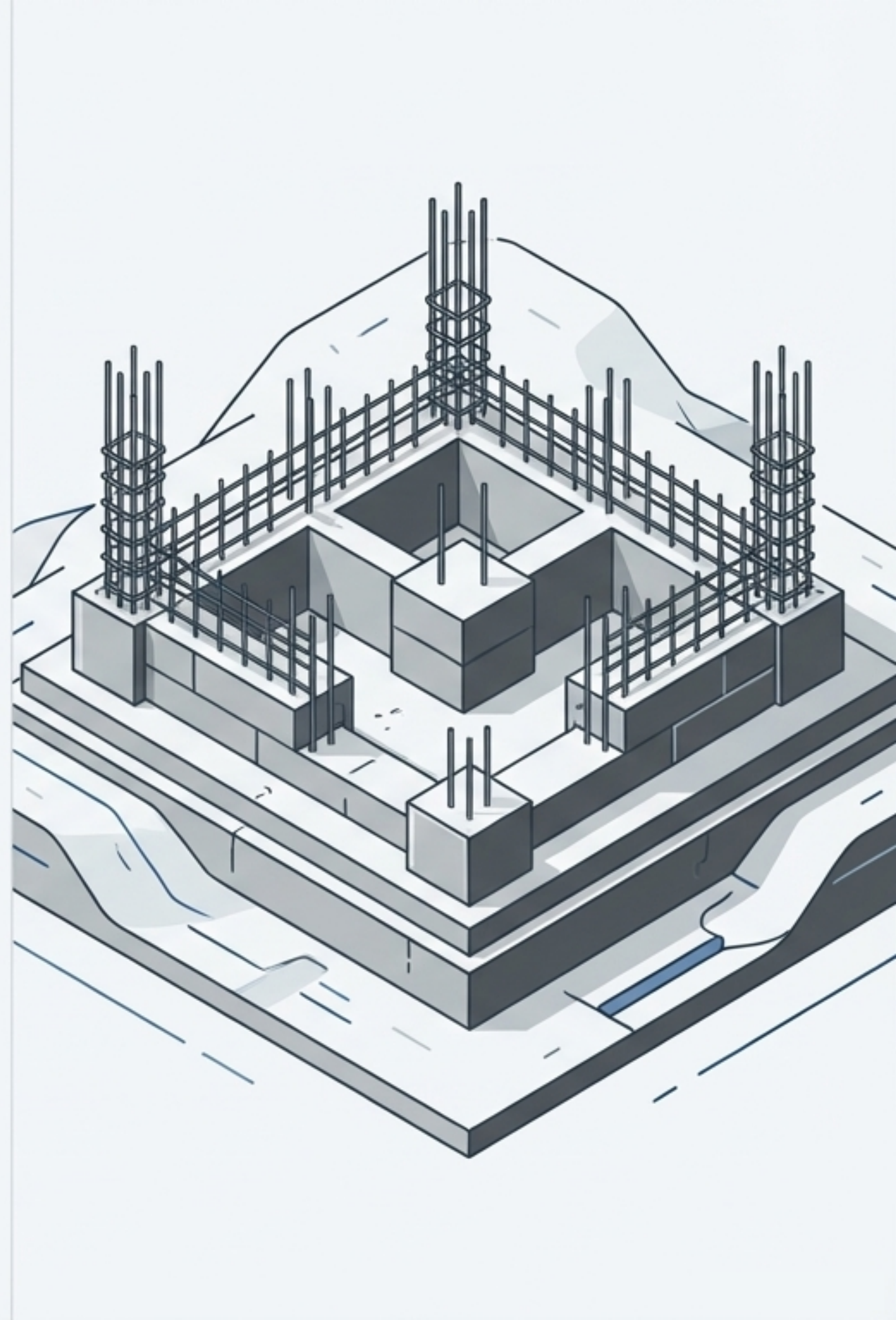


# CHAPITRE 2 : Établir une Base Permanente

## Persistance des données avec les Volumes Docker

### Objectifs

- 🎯 Créer un espace de stockage persistant géré par Docker (un Volume).
- 🎯 Lier ce volume à un conteneur pour y stocker les fichiers du site.
- 🎯 Vérifier que les données survivent à la suppression du conteneur.





# Connecter une fondation permanente à notre conteneur

## Étape 1: Créer le Volume

Un volume est un espace de stockage géré par Docker, indépendant du cycle de vie du conteneur.

```
docker volume create tp-volume
```

## Étape 2: Lancer un conteneur avec le Volume

Anatomy of the Command

```
docker run -it --name mon-php -p 8080:80  
-v tp-volume:/var/www/html php:8.2-apache bash
```

-v tp-volume:/var/www/html: **Monter le volume.** Connecte notre volume nommé `tp-volume` au dossier `/var/www/html` à l'intérieur du conteneur. Tout ce qui est écrit dans ce dossier sera en réalité stocké dans le volume.



# La Preuve de la Persistance



## Configuration Initiale

1. Dans le conteneur, créez le fichier `index.php` comme au Chapitre 1.
2. Quittez, puis supprimez brutalement le conteneur :

```
docker rm -f mon-php
```



## Vérification

1. Relancez un NOUVEAU conteneur en le liant au MÊME volume :

```
docker run -it --name mon-php -p  
8080:80 -v tp-volume:/var/www/html  
php:8.2-apache bash
```

2. Listez le contenu du répertoire web :

```
ls /var/www/html
```

```
> index.php
```

**\*\*Le fichier est toujours là !\*\***

Les données dans le volume ont survécu à la destruction du conteneur.



# CHAPITRE 3 :

## Construire le Pont de Développement

### Développement local en temps réel avec les Bind Mounts

- 🎯 Monter un dossier de votre machine locale directement dans le conteneur.
- 🎯 Modifier les fichiers de code avec vos outils habituels (VS Code, etc.).
- 🎯 Voir les changements reflétés instantanément dans l'application conteneurisée.





# Lier votre espace de travail local au conteneur

## Étape 1: Préparer le dossier de travail sur votre machine (Hôte)

```
mkdir tp-docker-site  
cd tp-docker-site  
echo "<?php echo '<h1>Bonjour depuis un bind mount</h1>'; ?>" > index.php
```

## Étape 2: Lancer le conteneur avec un Bind Mount

Anatomy of the Command



```
# Assurez-vous d'être dans le dossier tp-docker-site  
docker run -it --name mon-php -p 8080:80 -v "$(pwd)":/var/www/html php:8.2-apache bash
```

**-v "\$(pwd)":/var/www/html: Monter un dossier local.** Le chemin **\$(pwd)** (votre dossier courant) est directement "miroité" dans le dossier **/var/www/html** du conteneur. Ce n'est plus Docker qui gère le stockage, c'est un lien direct.



# Le Plan de l'Architecte : Choisir la Bonne Méthode

## Conteneur Seul



Persistance des Données:

✗ Non

Modification depuis l'Hôte:

✗ Non

Cas d'Usage Idéal:

Tests rapides,  
environnements jetables.

## Avec Volume



Persistance des Données:

✓ Oui

Modification depuis l'Hôte:

✗ Non (difficile)

Cas d'Usage Idéal:

Stockage de données de  
production (bases de  
données, uploads, etc.).

## Avec Bind Mount



Persistance des Données:

✓ Oui

Modification depuis l'Hôte:

✓ Oui (en temps réel)

Cas d'Usage Idéal:

Développement local.



# Commandes Utiles à Garder

## Gérer les Conteneurs

`docker ps`: Voir les conteneurs actifs.

`docker ps -a`: Voir TOUS les conteneurs (même arrêtés).

`docker stop [NAME]`: Arrêter un conteneur.

`docker rm [NAME]`: Supprimer un conteneur.

## Gérer les Volumes

`docker volume ls`: Lister les volumes existants.

`docker volume inspect [NAME]`: Obtenir les détails d'un volume.

`docker volume rm [NAME]`: Supprimer un volume (s'il n'est pas utilisé).

## Interagir

`docker exec -it [NAME] bash`: Ouvrir un shell dans un conteneur actif.



# Mission Accomplie !

Bravo ! Vous avez débloqué les compétences fondamentales de Docker.

- ✓ Lancer un conteneur Apache/PHP.
- ✓ Accéder à une application via un navigateur.
- ✓ Conserver des fichiers de manière permanente via un volume.
- ✓ Mettre en place un environnement de développement efficace avec un bind mount.

La **Suite de l'Aventure...** Votre prochaine étape pour automatiser et orchestrer vos environnements :

